

GATILHOS EM SQL (TRIGGERS)

Exercícios Adicionais de Prática com Gabarito Detalhado

Após realizar os primeiros testes, você percebeu a importância de manter as chaves estrangeiras ativas no SQLite usando `PRAGMA foreign_keys = ON;`. Para aprofundar seus conhecimentos na automatização e proteção do banco de dados, preparamos esta **lista complementar de 5 exercícios focados exclusivamente em Triggers (Gatilhos)**. Estes exercícios cobrem desde auditorias simples até regras de negócios e validações complexas que ajudam a manter a integridade dos seus dados.

📌 **LEMBRETE IMPORTANTE:** Assim como no museu (MFA) abordado por Carter Zanke na aula do CS50, os gatilhos nos dão acesso aos modificadores **OLD** (estado da linha antes da ação, disponível em `UPDATES` e `DELETES`) e **NEW** (estado da linha após a ação, disponível em `INSERTS` e `UPDATES`). Use-os com sabedoria!

PARTE 1: ENUNCIADO DOS EXERCÍCIOS

Exercício 1: Registro Automático de Remoção (Dificuldade: Fácil)

Cenário: Quando um livro é fisicamente excluído do acervo para limpeza física, o sistema deve guardar um rastro dessa exclusão.

Tarefa: Projete um trigger chamado `log_remocao_livro` que seja disparado **após** a exclusão física (`DELETE`) de qualquer linha da tabela `livros`. O trigger deve inserir um registro na tabela `historico_vendas` salvando o título do livro excluído, o nome da ação como **'REMOVIDO'**, e a data do registro como a data atual.

Exercício 2: Validação de Regras de Negócio com Erro (Dificuldade: Média)

Cenário: Vendas com quantidade negativa ou igual a zero representam inconsistência grave no fluxo financeiro.

Tarefa: Crie um trigger chamado `validar_quantidade_venda` que seja executado **antes** de qualquer inserção (`BEFORE INSERT`) na tabela `vendas`. Se o valor da coluna `quantidade` da nova venda for menor ou igual a zero, o trigger deve cancelar a transação usando a função `RAISE(FAIL, ...)` com a mensagem: **'A quantidade vendida deve ser maior que zero!'**.

Exercício 3: Histórico de Alterações de Preço (Dificuldade: Média)

Cenário: Os preços dos livros flutuam com promoções e inflação. Precisamos auditar essas mudanças.

Tarefa: Primeiro, crie uma tabela de auditoria chamada `historico_precos` contendo as colunas `id` (Primary Key), `livro_id` (Foreign Key referenciando `livros`), `preco_antigo`, `preco_novo` e `data_alteracao`.

Depois, crie um trigger chamado `log_alteracao_preco` que dispare **após** qualquer atualização de dados (AFTER UPDATE) **especificamente na coluna** `preco` da tabela `livros`. O trigger deve popular a tabela de auditoria, garantindo (através de uma cláusula WHEN) que o log só seja inserido se o preço novo for de fato diferente do preço antigo.

Exercício 4: Impedir Exclusão de Autor Ativo (Dificuldade: Difícil)

Cenário: Deletar um autor do banco que ainda possui livros ativos cria sérios problemas operacionais, mesmo que as chaves estrangeiras evitem erros estruturais, queremos uma regra lógica explícita.

Tarefa: Crie um trigger de segurança chamado `impedir_delecao_autor_ativo` que execute **antes** de uma exclusão (BEFORE DELETE) na tabela `autores`. O trigger deve impedir a exclusão se o autor possuir pelo menos um livro ativo na livreria (isto é, um livro associado na tabela `autoria` cujo campo `deletado` na tabela `livros` seja igual a 0). Se houver livro ativo, lance um erro usando `RAISE(FAIL, 'Não é possível deletar um autor que possui livros ativos!')`.

Exercício 5: Log de Estado no Soft Delete (Dificuldade: Avançada)

Cenário: Na nossa livreria, usamos Soft Delete. Quando um livro é excluído logicamente (seu campo `deletado` passa de 0 para 1), precisamos registrar essa desativação no histórico de vendas.

Tarefa: Escreva um trigger chamado `log_soft_delete_livro` que dispare **após** a atualização do campo `deletado` (AFTER UPDATE OF `deletado`) na tabela `livros`. O trigger deve rodar apenas se o novo valor de `deletado` for igual a 1 e o valor antigo for igual a 0. Ele deve inserir um registro em `historico_vendas` com o título do livro, a ação sendo **'DESATIVADO'** e a data atual.

PARTE 2: GABARITO E EXPLICAÇÃO DETALHADA

Resolução do Exercício 1:

Como a linha está sendo excluída física do banco de dados, usamos a cláusula **AFTER DELETE**. Não existe uma linha 'nova' neste contexto, então fazemos referência às colunas antigas usando o modificador **OLD**.

```
CREATE TRIGGER log_remocao_livro
AFTER DELETE ON livros
FOR EACH ROW
BEGIN
    INSERT INTO historico_vendas (titulo_livro, acao, data_registro)
    VALUES (OLD.titulo, 'REMOVIDO', strftime('%Y-%m-%d', 'now'));
END;
```

Resolução do Exercício 2:

Para validar e bloquear inserções incorretas antes que entrem no banco, usamos **BEFORE INSERT**. O modificador **NEW** representa a linha que está tentando ser gravada. O comando **RAISE(FAIL, ...)** interrompe a operação imediatamente e reverte qualquer alteração.

```
CREATE TRIGGER validar_quantidade_venda
BEFORE INSERT ON vendas
FOR EACH ROW
BEGIN
    SELECT RAISE(FAIL, 'A quantidade vendida deve ser maior que zero!')
    WHERE NEW.quantidade <= 0;
END;
```

Resolução do Exercício 3:

Aqui combinamos a criação da tabela de auditoria com um trigger focado em atualizações. A cláusula **AFTER UPDATE OF preco ON livros** restringe o gatilho apenas para alterações de preço. A cláusula **WHEN** evita logs se o preço foi sobrescrito pelo mesmo valor.

```
-- 1. Criando a tabela de auditoria de preços
CREATE TABLE historico_precos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    livro_id INTEGER,
    preco_antigo REAL,
    preco_novo REAL,
    data_alteracao TEXT,
    FOREIGN KEY (livro_id) REFERENCES livros(id)
);

-- 2. Criando o trigger com condicional WHEN
CREATE TRIGGER log_alteracao_preco
AFTER UPDATE OF preco ON livros
FOR EACH ROW
WHEN OLD.preco <> NEW.preco
BEGIN
    INSERT INTO historico_precos (livro_id, preco_antigo, preco_novo, data_alteracao)
    VALUES (OLD.id, OLD.preco, NEW.preco, strftime('%Y-%m-%d %H:%M:%S', 'now'));
END;
```

Resolução do Exercício 4:

Este gatilho **BEFORE DELETE** é uma excelente forma de proteger a integridade lógica de negócios. Usamos uma subquery com a cláusula EXISTS para verificar se o autor que está tentando ser removido possui livros vinculados na tabela associativa que continuem com deletado = 0.

```
CREATE TRIGGER impedir_delecao_autor_ativo
BEFORE DELETE ON autores
FOR EACH ROW
BEGIN
    SELECT RAISE(FAIL, 'Não é possível deletar um autor que possui livros ativos!')
    WHERE EXISTS (
        SELECT 1 FROM autoria a
        JOIN livros l ON a.livro_id = l.id
        WHERE a.autor_id = OLD.id AND l.deletado = 0
    );
END;
```

Resolução do Exercício 5:

Este exercício ensina a detectar transições de estado específicas em 'Soft Delete'. Queremos interceptar apenas quando o livro for marcado como deletado (OLD.deletado = 0 e NEW.deletado = 1). Como estamos alterando dados existentes, usamos **AFTER UPDATE OF**.

```
CREATE TRIGGER log_soft_delete_livro
AFTER UPDATE OF deletado ON livros
FOR EACH ROW
WHEN NEW.deletado = 1 AND OLD.deletado = 0
BEGIN
    INSERT INTO historico_vendas (titulo_livro, acao, data_registro)
    VALUES (NEW.titulo, 'DESATIVADO', strftime('%Y-%m-%d', 'now'));
END;
```

▮ **PRÓXIMOS PASSOS DE ESTUDO:** Agora que você viu como automatizar ações e impor regras rígidas de integridade, tente copiar estes códigos SQL e rodar no seu terminal do SQLite conectado ao banco `pratica_sql.db`. Experimente fazer inserts ou updates de teste para ver os gatilhos dispararem e as tabelas de histórico serem populadas sozinhas. Bons estudos!